StitchCheck — Alibaba x Atlas Hackathon "Must Do Now" Plan (Plain-Language Explainer)

Saved for reference. Personal internal deadline: 29 Aug 2026, 02:00 SGT (ahead of the official 30 Aug 23:59 SGT deadline, to leave room for the Daytona HackSprint later on 29 Aug).

The core idea

Right now, StitchCheck only does one simple thing: it notices "this connection is risky" and shows some alternative flights. The judges have said that's the boring, common answer everyone will submit. So tonight's job is to make StitchCheck look **one level smarter**, without doing anything risky like real payments or bookings.

What "dependency-graph re-planning" actually means

Think of a trip like a row of dominoes:

```
First flight delayed
   → you miss your connecting flight
      → your hotel check-in is at risk
         → your whole plan falls apart
```

Right now, the app only looks at the first domino falling. The upgrade makes it look at the **whole chain at once** — and instead of just listing random alternative flights, it picks **one single replacement plan** that fixes the entire chain, not just one leg.

That's the "wow moment": on screen, you'll visually see the delayed flight turn red, then watch the downstream flights/hotel also turn red (showing the ripple effect), and then the app collapses all that mess into **one recommended fix** for the entire trip.

Why this matters for scoring

The organizers explicitly said:

- "Detect a delay, suggest alternatives" = the boring answer everyone submits.
- "Treat the itinerary as a connected system and re-plan the whole thing" = the impressive answer.

So this single change is aimed at jumping from "boring" to "impressive" without needing anything dangerous like handling real money.

The safety rules that must stay in place

Even though the app looks smarter, it must **never pretend** to have actually booked or paid for anything. So:

- The traveler can click "Review recovery plan," then "Confirm switch request" — but that's just a *request*, not an actual booking.
- The app must never say "Booked" or "Switched" unless it actually got real confirmation back from Atlas. Otherwise it just says "Request submitted — awaiting verified outcome."
- If the app can't find a safe new plan after two tries, it should honestly say "No safe plan found — escalate to a human," instead of looping forever pretending to think.
- Right before asking for confirmation, it should show "Availability refreshed just now" with a timestamp — proving it double-checked the latest information instead of using old, possibly wrong data.

These three protections map to three specific mistakes the judges are watching for: apps that loop forever without giving up, apps that use stale/outdated information, and apps that falsely claim success without checking if anything actually worked.

The video "wow moment" script, in plain words

The narration basically says: *"A missed connection isn't just one problem — it breaks your whole trip. Our AI looks at the entire chain of consequences and gives you one safe fix, but you're always the one who approves it before anything happens."*

The safety net (fallback plan)

There's a hard deadline check: **90 minutes before your 2 AM cutoff**, you must decide — is the fancy new feature actually working and ready to film?

- **If yes:** film the impressive new version.
- **If no:** don't panic or rush a broken feature on camera. Just film the simpler, already-working version (extract flight info → correct it → show the risk → show alternatives → confirm). It's honest, safe, and still solid — better to submit something small that clearly works than something ambitious that visibly breaks on camera.

Things to deliberately NOT touch tonight

To avoid wasting time or introducing risk:

- Don't build refund/cancellation/payment features.
- Don't try real bookings, even test ones, unless already fully proven to work.
- Don't rebuild your video recording pipeline — reuse what already works.
- Don't let the AI pretend to make payment or refund decisions on its own — that's explicitly penalized by the judges.
- Don't build a big new system architecture from scratch.

The "proof of work" tricks (cheap evidence to grab)

Since part of your score depends on showing you actually used the Qoder tool properly, grab a few screenshots that cost almost no time:

- A screenshot showing your task/plan inside Qoder moving from "planned" to "built."
- A screenshot showing your existing automated tests passing.
- Just link to documents you already wrote earlier (PRD, spec) instead of writing new ones.
- If possible, a screenshot showing you used a cheaper/faster AI model for routine coding and saved the more powerful model only for the harder "figure out the recovery plan" logic — this shows you're being cost-conscious, which is also graded.

Final 90 minutes = no more coding

In the very last stretch before your deadline, stop writing code entirely and just:

1. Record one clean 3-minute video take.
2. Check the video plays correctly with working sound (`ffprobe` is just a tool that checks the video file is valid).
3. Make sure every label is honest — nothing that looks "live" is actually just a demo/fake example, and it's clearly marked as such.
4. Re-run your existing automated checks and save proof they passed.
5. Drop your evidence screenshots into the project folder.
6. Make sure there are no passwords/secret keys anywhere in the code you're submitting.
7. Submit the video and form, then keep a backup copy of everything together.

Bottom line

Build one smart feature (whole-trip re-planning instead of single-flight alerts), never claim to actually book/pay for anything, prove you have safety checks, grab cheap evidence you used the tool properly, and have a safe fallback ready if you run out of time.